

# BASH SCRIPTING

# Linux Shells Overview

- **Linux** has a variety of different **shells**:
  - Bourne Shell (sh)
  - C Shell (csh)
  - Korn Shell (ksh)
  - TC Shell (tcsh)
  - **Bourne Again Shell (bash)**
  - **Z Shell (zsh)**
  - ...
- Certainly, the most popular shell is “bash”. Bash is an sh-compatible shell that incorporates useful **features** from the **Korn** shell (ksh) and C shell (csh)
- It is intended to conform to the *IEEE POSIX P1003.2/ISO 9945.2* Shell and Tools standard.
- It offers functional **improvements** over sh for both programming and interactive use

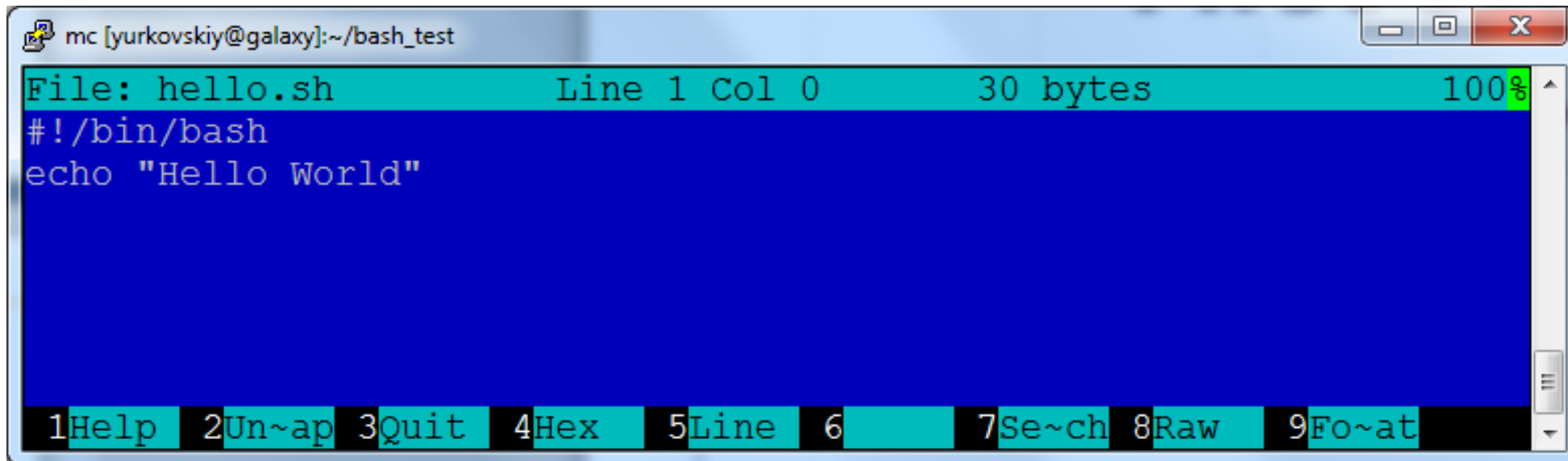


# BASH Syntax

- Latin alphabet
- Arabic digits
- Punctuation symbols
- Some keywords
- ...

# BASH Syntax

- On the first line
- `#!/bin/bash`
- Tells the operating system that the following will be a script and not a regular text file



The screenshot shows a text editor window titled 'mc [yurkovskiy@galaxy]:~/bash\_test'. The editor displays a file named 'hello.sh' with the following content:

```
File: hello.sh      Line 1 Col 0      30 bytes      100%  
#!/bin/bash  
echo "Hello World"
```

The editor interface includes a status bar at the bottom with various menu options: 1Help, 2Un~ap, 3Quit, 4Hex, 5Line, 6, 7Se~ch, 8Raw, 9Fo~at.

softserve

# VARIABLES (1)

- We can use **variables** as in any programming languages. Their values are **always stored as strings**, but there are mathematical operators in the shell language that will *convert variables to numbers for calculations*.
- There is no needed to **declare** variables. Just assign a value to its reference will create it
- How to use variables

```
mc [yurkovskiy@galaxy]:~/bash_test
File: second.sh      Line 1 Col 0
#!/bin/bash
CDIR="BASH_TRASH"
echo "Creating directory...$CDIR"
mkdir $CDIR
cp * $CDIR
rm -rf $CDIR
```

1Help 2Un~ap 3Quit 4Hex 5Line 6 7Se~ch 8Raw

Value assignment and  
variable declaration

Using Variable's value

softserve

# VARIABLES (2)

- The shell programming language does **not type-cast** its variables.
  - This means that a variable can hold **number** data or **character** data
- Switching the **TYPE** of a variable can lead **to confusion** for the writer of the script or someone trying to modify it, so **it is recommended to use a variable for only a single TYPE of data** in a script
- **\** is the bash **escape** character and it **preserves** the **literal value** of the next character that follows

```
count=0
```

```
count=Sunday
```

**softserve**

# VARIABLES (3)

- When assigning character data containing *spaces or special characters*, the data must be enclosed in either **single or double quotes**.
- Using *double quotes* to show a string of characters will allow any variables in the quotes to be *resolved*.
- Using *single quotes* to show a string of characters will *not allow variable resolution*.

```
var="test string"  
newvar="Value of var is $var"  
echo $newvar
```

```
var='test string'  
newvar='Value of var is $var'  
echo $newvar
```

# ENVIRONMENTAL VARIABLES (1)

- There are two types of variables:
  - Local Variables
  - Environmental Variables
- **Environmental variables** are set by the system and can usually be found by using the **env** command. Environmental variables hold special values
- Environmental variables are **defined** in */etc/profile*, */etc/profile.d/* and *~/.bash\_profile*. These files are the **initialization** files, and they are read when bash shell is invoked.
- When a login shell exits, bash reads *~/.bash\_logout*.
- The **startup** is more complex; for example, if bash is used interactively, then */etc/bashrc* or *~/.bashrc* are read.
  - See the man page for more details.

**softserve**



# ENVIRONMENTAL VARIABLES (2)

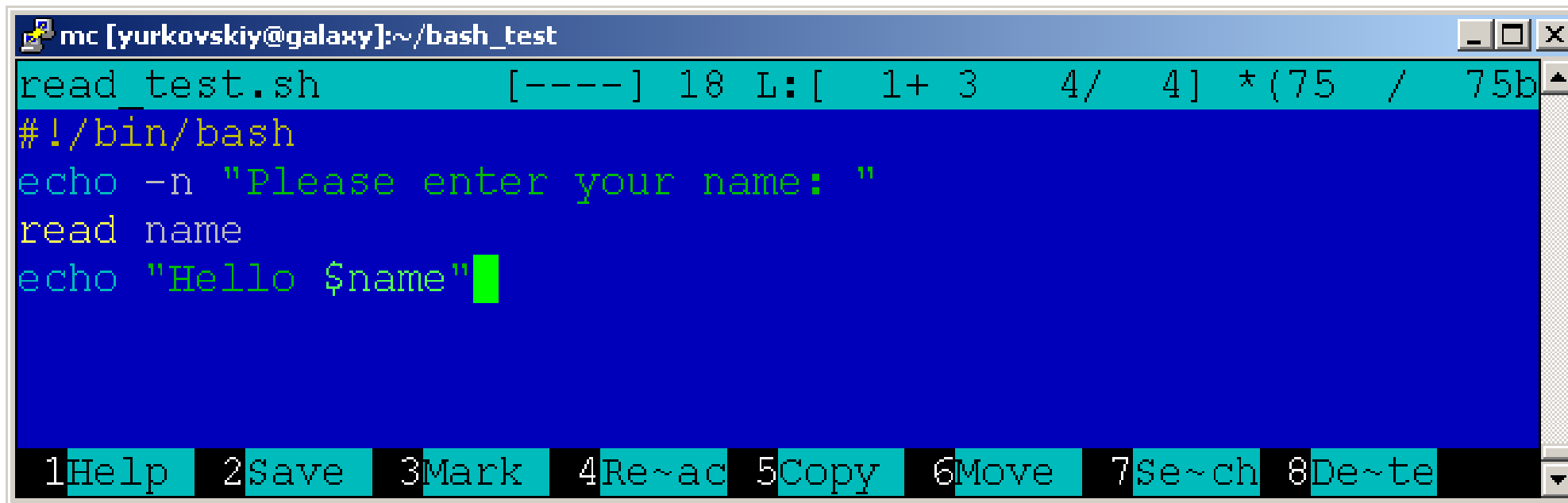
| Name                 | Description                                                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>PATH</b>          | The search path for commands. It is a <a href="#">colon-separated</a> list of directories that are searched when you type a command |
| <b>HOSTNAME</b>      | Name of the host (computer)                                                                                                         |
| <b>USER, LOGNAME</b> | Current logged in user                                                                                                              |
| <b>PS1</b>           | Sequence of characters shown before the prompt                                                                                      |
| <b>PWD</b>           | Current working directory                                                                                                           |
| <b>SHELL</b>         | The path to the current command shell                                                                                               |
| ...                  |                                                                                                                                     |

# ENVIRONMENTAL VARIABLES (1)

- PS1: sequence of characters shown before the prompt
  - \t – hour
  - \d – date
  - \w – current directory
  - \W – last part of current directory
  - \u – user name
  - \\$ - prompt character

# VARIABLES

- The read command allows you to prompt for input and store it in a variable



```
mc [yurkovskiy@galaxy]:~/bash_test
read_test.sh      [-----] 18 L: [ 1+ 3 4/ 4] *(75 / 75b
#!/bin/bash
echo -n "Please enter your name: "
read name
echo "Hello $name"
```

1Help 2Save 3Mark 4Re~ac 5Copy 6Move 7Se~ch 8De~te

softserve

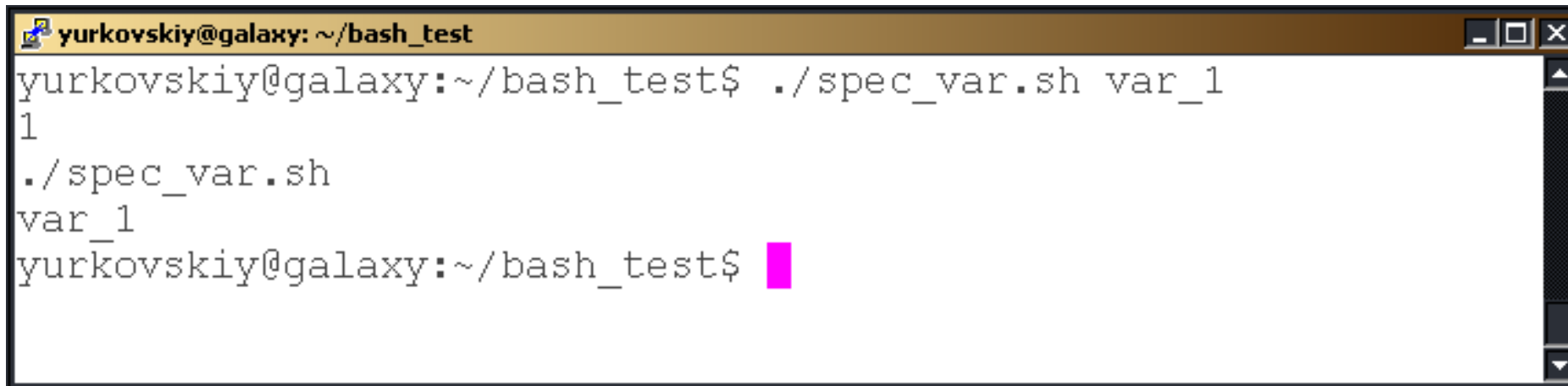
# SPECIAL VARIABLES

| Parameter | Description                                          |
|-----------|------------------------------------------------------|
| \$0       | Name of the current shell script                     |
| \$1-\$9   | Positional parameters 1 through 9                    |
| \${10}    | Positional parameter 10                              |
| \$#       | The number of positional parameters                  |
| \$*       | All positional parameters, "\$*" is one string       |
| \$@       | All positional parameters, "\$@" is a set of strings |
| \$?       | Return status of most recently executed command      |
| \$\$      | Process id of current process                        |

**softserve**

# ENVIRONMENTAL VARIABLES

- We have a next bash script
  - `#!/bin/bash`
  - `echo $#`
  - `echo $0`
  - `echo $1`

A terminal window with a title bar that reads "yurkovskiy@galaxy: ~/bash\_test". The terminal shows the command `./spec_var.sh var_1` being executed. The output consists of three lines: `1`, `./spec_var.sh`, and `var_1`. The prompt `yurkovskiy@galaxy:~/bash_test$` is followed by a red cursor.

```
yurkovskiy@galaxy: ~/bash_test
yurkovskiy@galaxy:~/bash_test$ ./spec_var.sh var_1
1
./spec_var.sh
var_1
yurkovskiy@galaxy:~/bash_test$
```

**softserve**

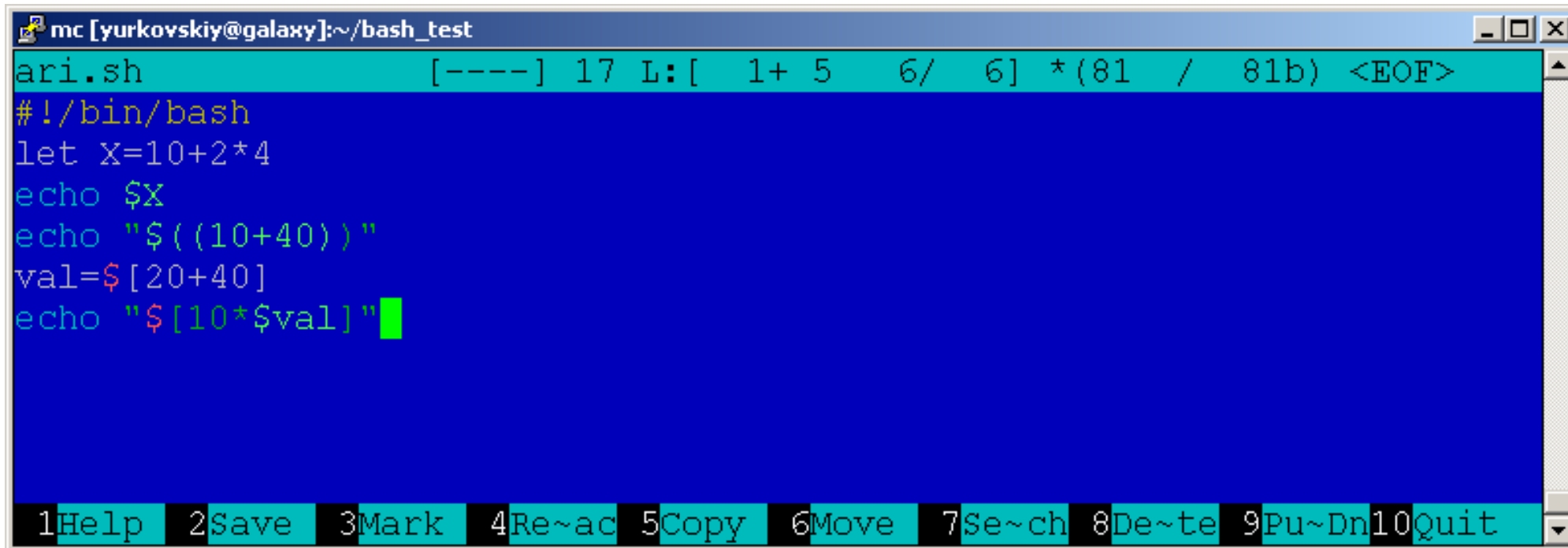
# COMMAND SUBSTITUTION

- The **backquote** “```” is different from the **single quote** “`'`”.
- It is used for **command substitution**: ``command``

```
#!/bin/bash  
list=`ls -l`  
echo $list
```

# ARITHMETIC EVALUATION

- The **let** statement can be used to do mathematical functions:
- An arithmetic expression can be evaluated by **`$(expression)`** or **`$((expression))`**
- ATTENTION, PLEASE BASH DON'T KNOW HOW TO WORK WITH **floating** point units ☺



```
mc [yurkovskiy@galaxy]:~/bash_test
ari.sh [-----] 17 L: [ 1+ 5 6/ 6] *(81 / 81b) <EOF>
#!/bin/bash
let X=10+2*4
echo $X
echo "$((10+40))"
val=$((20+40))
echo "$[10*$val]"
```

The screenshot shows a terminal window titled 'mc [yurkovskiy@galaxy]:~/bash\_test'. The prompt is 'ari.sh [-----] 17 L: [ 1+ 5 6/ 6] \*(81 / 81b) <EOF>'. The terminal content is as follows:

```
#!/bin/bash
let X=10+2*4
echo $X
echo "$((10+40))"
val=$((20+40))
echo "$[10*$val]"
```

At the bottom of the terminal window, there is a menu bar with the following options: 1Help, 2Save, 3Mark, 4Re~ac, 5Copy, 6Move, 7Se~ch, 8De~te, 9Pu~Dn, 10Quit.

softserve

# ARITHMETIC EVALUATION

```
#!/bin/bash
echo $(( 100 / 3 ))
myvar="56"
echo $(( $myvar + 12 ))
echo $(( $myvar - $myvar ))
myvar=$(( $myvar + 1 ))
echo $myvar
```



# CONDITIONAL STATEMENTS

- **Conditionals** let us decide whether to perform an action or not, this decision is taken by evaluating an expression. The most basic form is:

```
if [ expression ];  
    then  
        statements  
    elif [ expression ];  
    then  
        statements  
    else  
        statements  
fi
```

- the **elif** (else if) and **else** sections are optional **softserve**
- Put **spaces** after [ and before ], and around the operators and operands.

# EXPRESSION (1)

- An expression can be:
  - **String** comparison
  - **Numeric** comparison
  - File operators and Logical operators and it is represented by [expression]:
- String Comparisons:

| Expression | Description                                           |
|------------|-------------------------------------------------------|
| =          | compare if two strings are equal                      |
| !=         | compare if two strings are not equal                  |
| -n         | evaluate if string <b>length</b> is greater than zero |
| -z         | evaluate if string length is equal to zero            |

# EXPRESSION (2)

- Number Comparisons

| Expression | Description                                                       |
|------------|-------------------------------------------------------------------|
| <b>-eq</b> | compare if two numbers are <b>equal</b>                           |
| <b>-ge</b> | compare if one number is <b>greater than or equal</b> to a number |
| <b>-le</b> | compare if one number is <b>less than or equal</b> to a number    |
| <b>-ne</b> | compare if two numbers are <b>not equal</b>                       |
| <b>-gt</b> | compare if one number is <b>greater</b> than another number       |
| <b>-lt</b> | compare if one number is <b>less</b> than another number          |

# EXPRESSION

```
if [ "$myvar" -eq 3 ]  
    then echo "myvar eq 3 as num"  
fi
```

```
if [ "$myvar" = "3" ]  
    then echo "myvar eq 3 as srt"  
fi
```

```
if [ -z $1 ]; then echo "Empty Parameter"; fi
```

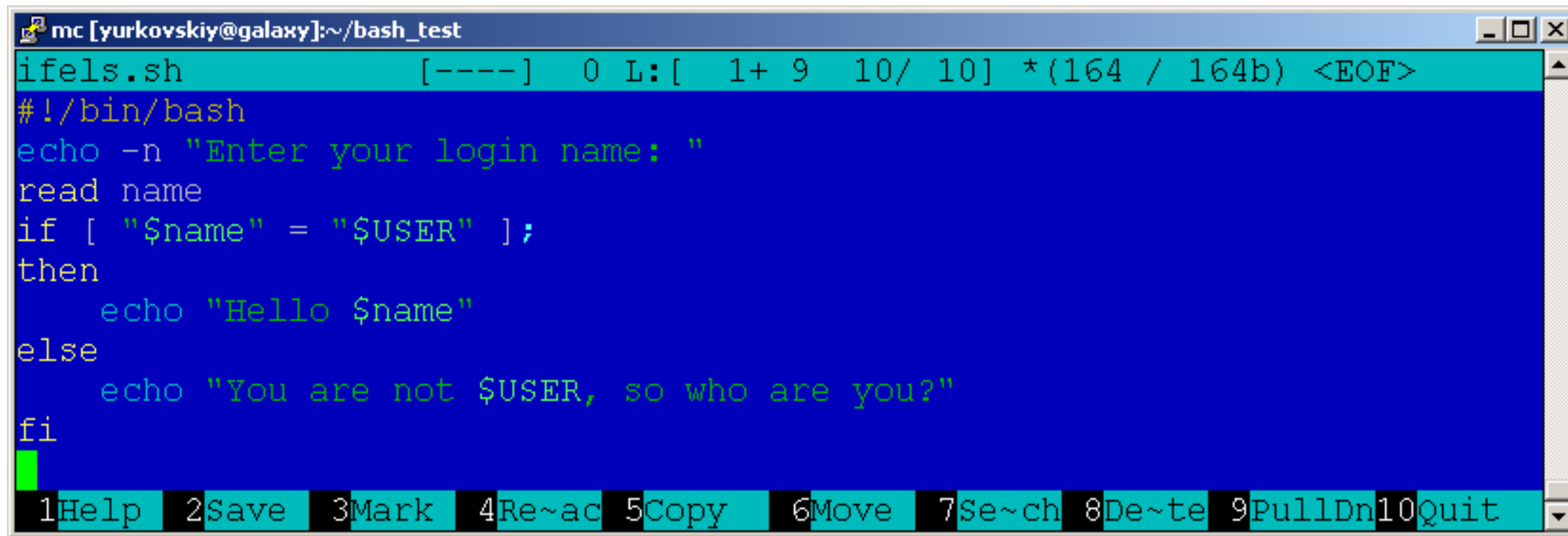
# RELATIONAL OPERATORS

| Meaning                            | Numeric | String      |
|------------------------------------|---------|-------------|
| Greater than                       | -gt     |             |
| Greater than or equal              | -ge     |             |
| Less than                          | -lt     |             |
| Less than or equal                 | -le     |             |
| Equal                              | -eg     | = or ==     |
| Not equal                          | -ne     | !=          |
| str1 is less than str2             |         | str1 < str2 |
| str1 is greater str2               |         | str1 > str2 |
| String length is greater than zero |         | -n str      |
| String length is zero              |         | -z str      |

**softserve**

# RELATIONAL OPERATORS

- Script

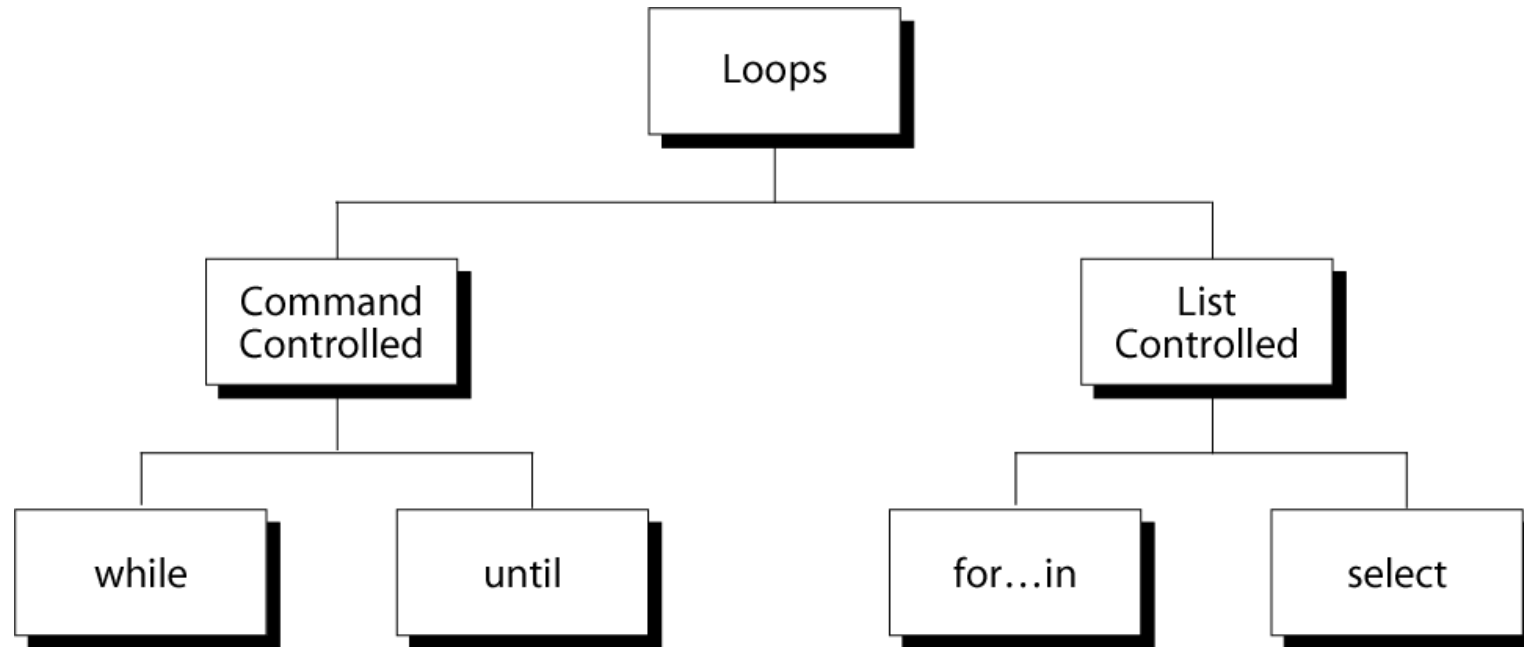


```
mc [yurkovskiy@galaxy]:~/bash_test
ifels.sh      [-----]  0 L:[  1+ 9  10/ 10] *(164 / 164b) <EOF>
#!/bin/bash
echo -n "Enter your login name: "
read name
if [ "$name" = "$USER" ];
then
    echo "Hello $name"
else
    echo "You are not $USER, so who are you?"
fi
```

softserve

# LANGUAGE ELEMENTS

- Control structures
  - Repetition
    - do-while, repeat-until
    - for ... in
    - Select
  - Functions
  - Trapping signals



**softserve**

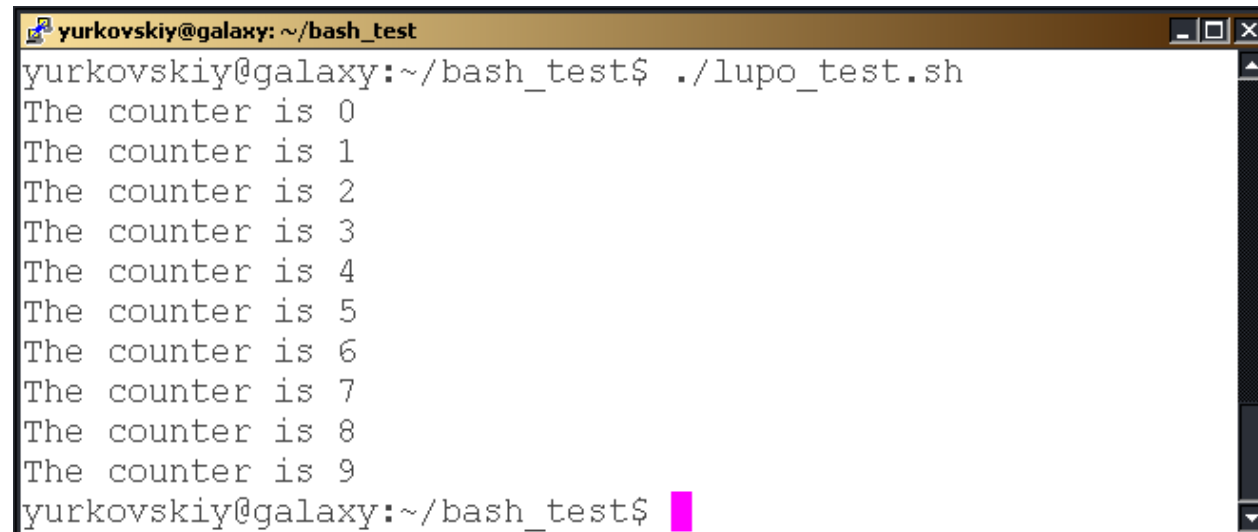
# LANGUAGE ELEMENTS

- Purpose:
- To execute commands in “command-list” as long as “expression” evaluates to true
- Syntax:
  - `while [ expression ]`
  - `do`
  - `command-list`
  - `done`



# LANGUAGE ELEMENTS

```
#!/bin/bash
COUNTER=0
while [ $COUNTER -lt 10 ]
do
    echo "The counter is $COUNTER"
    let COUNTER=$COUNTER+1
done
```

A terminal window titled 'yurkovskiy@galaxy: ~/bash\_test' showing the execution of a script. The prompt is 'yurkovskiy@galaxy:~/bash\_test\$ ./lupo\_test.sh'. The output consists of ten lines: 'The counter is 0' through 'The counter is 9'. The prompt 'yurkovskiy@galaxy:~/bash\_test\$' is visible at the bottom with a pink cursor.

```
yurkovskiy@galaxy: ~/bash_test
yurkovskiy@galaxy:~/bash_test$ ./lupo_test.sh
The counter is 0
The counter is 1
The counter is 2
The counter is 3
The counter is 4
The counter is 5
The counter is 6
The counter is 7
The counter is 8
The counter is 9
yurkovskiy@galaxy:~/bash_test$
```

**softserve**

# LANGUAGE ELEMENTS

```
#!/bin/bash
```

```
for x in one two three three four; do  
    echo "number $x"  
done
```

```
#!/bin/bash
```

```
for myfile in /etc/r*; do  
    if [ -d "$myfile" ]  
        then echo "$myfile (dir) "  
    else  
        echo "$myfile"  
    fi  
done
```

**softserve**

# LANGUAGE ELEMENTS

- Repeat until true

```
myvar=0
while [ $myvar -ne 10 ]; do
    echo "$myvar"
    myvar=$(( $myvar + 1 ))
done
```

- Repeat until the value is false

```
myvar=0
until [ $myvar -eq 10 ]
do
    echo $myvar
    myvar=$(( $myvar + 1 ))
done
```

# ARRAY

A **bash array** is a data structure designed to store information in an indexed way. It is extra useful if users need to store and retrieve thousands of pieces of data fast. What makes bash arrays special is that unlike any other programming language, they can **store different types of elements**. For example, you can use a bash array to store both strings and numbers.

```
#!/bin/bash
# Create an indexed array
IndexedArray=(egg burger milk)
#Iterate over the array to get all the values
for i in "${IndexedArray[@]}";do echo "$i";done
```

# STRING

## Display String Length

```
#!/bin/bash  
# Create a new string  
mystring="lets count the length of this string"  
i=${#mystring}  
echo "Length: $i"
```

Here, the **#** operator is used to get the length of the string variable

# EXTRACT STRING

If users need to remove unnecessary parts from strings, they can use the Bash string extraction tools.

The following script has 4 values, 3 of them being strings. In our example, we will extract only the number value. This can be done via the cut command. First, we instruct the command that each variable is separated by a comma by using the -d flag. Then we ask the cut command to extract the 5th value.

```
#!/bin/bash
```

```
cut -d , -f 5 <<< "Website,Domain,DNS,SMTP,5005"
```

In another example, we have a string that is mixed with some numbers. We will use expr substr commands to extract only the SoftServe text value.

```
#!/bin/bash
```

```
expr substr "458449SoftServe4132" 7 9
```

**softserve**

# FIND AND REPLACE STRING

```
#!/bin/bash  
first="I like a DevOps and Python"  
second="Golang"  
echo "${first/Python/$second}"
```

```
$ ./frstr.sh  
I like a DevOps and Golang
```

The find and replace functionality doesn't require any special commands, it can all be done with string manipulation.

# FUNCTION

- Variable in function

```
#!/bin/bash
```

```
myvar="hello"
```

```
myfunc() {
```

```
    myvar="one two three"
```

```
    for x in $myvar; do
```

```
        echo $x
```

```
    done
```

```
}
```

```
myfunc
```

```
echo "\$myvar = $myvar  \$x = $x"
```

- Result one two three three



# FUNCTION

- A local variable in a function

```
#!/bin/bash
```

```
myvar="hello"
```

```
myfunc() {
```

```
    local x
```

```
    local myvar="one two three"
```

```
    for x in $myvar; do
```

```
        echo $x
```

```
    done
```

```
}
```

```
myfunc
```

```
echo "\$myvar = $myvar  \$x = $x"
```

- **Result** hello

**softserve**

# FUNCTION

## Create function with Parameters

Bash can't declare function parameter or arguments at the time of function declaration. But you can use parameters in function by using other variable.

```
#!/bin/bash
Rectangle_Area() {
area=$(( $1 * $2 ))
echo "Area is : $area"
}
Rectangle_Area 10 20
```

```
ubuntu $ touch 1.sh
ubuntu $ chmod 755 1.sh
ubuntu $ nano 1.sh
ubuntu $ ./1.sh
Area is:200
```

**softserve**

# Intro to Regular Expression

Since version 3 (circa 2004), bash has a built-in regular expression comparison operator, represented by `=~`. A lot of scripting tricks that use `grep` or `sed` can now be handled by bash expressions and the bash expressions might just give you scripts that are easier to read and maintain. As with other comparison operators (e.g., `-lt` or `==`), bash will return a zero if an expression like `$digit =~ "[0-9]"` shows that the variable on the left matches the expression on the right and a one otherwise. This example test asks whether the value of `$digit` matches a single digit.

```
if [[ $digit =~ [0-9] ]]; then
    echo "$digit is a digit"
else
    echo "oops"
fi
```

# CASE STUDY

Print Files With Line Count

lang.txt

C#

C++

Java

Python

JavaScript

Golang

PHP

Ruby

```
#!/bin/bash
myfile='lang.txt'
i=1
while read lines; do
echo "$i : $lines"
i=$((i+1))
done < $myfile
```

```
$ ./lang.sh
1 : C#
2 : C++
3 : Java
4 : Python
5 : JavaScript
6 : Golang
7 : PHP
8 : Ruby
```

**softserve**

# CASE STUDY

## Test if File Exists

lang.txt

C#

C++

Java

Python

JavaScript

Golang

PHP

Ruby

```
#!/bin/bash
MYFILE='lang.txt'
if [ -f "$MYFILE" ]; then
echo "$MYFILE exists."
else
echo "$MYFILE does not exist."
fi
```

```
$ ./exists.sh
lang.txt exists.
```

**softserve**

# CASE STUDY

## Show Server Information

### serverinf.sh

```
#!/bin/bash
```

```
echo "Date"
```

```
date
```

```
echo "Uptime"
```

```
uptime
```

```
echo "Memory Usage"
```

```
free -m
```

```
echo "Network Usage"
```

```
ip a
```

```
[root@localhost ~]# ./shinfo.sh
Date
Tue Jul 25 11:20:39 AM UTC 2023
Uptime
 11:20:40 up 5 min,  0 users,  load average: 0.00, 0.01, 0.00
Memory Usage

```

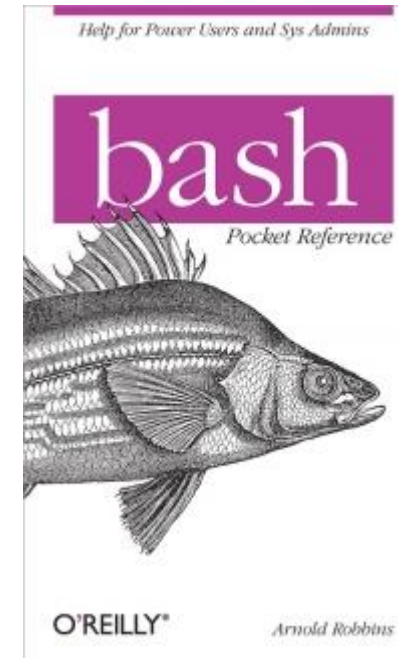
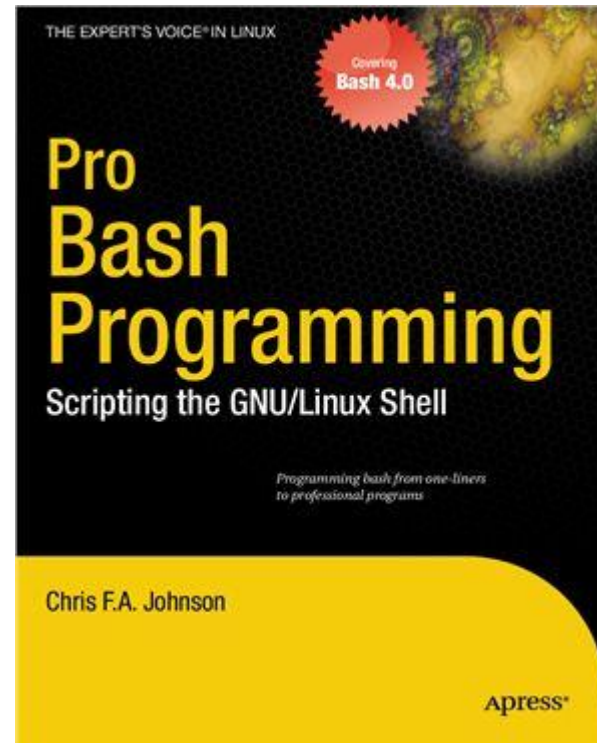
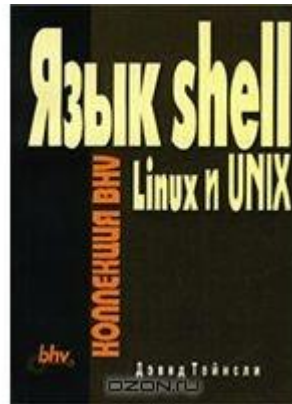
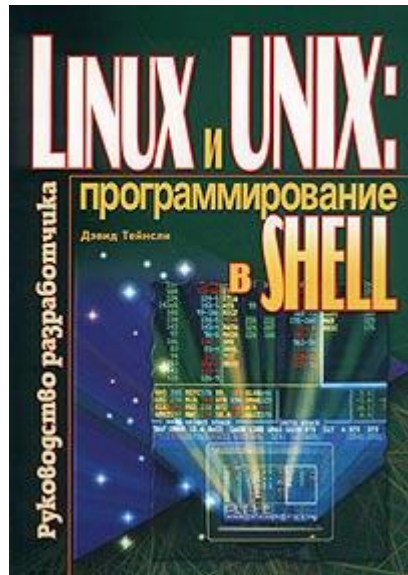
|       | total | used | free | shared | buff/cache | available |
|-------|-------|------|------|--------|------------|-----------|
| Mem:  | 181   | 5    | 171  | 0      | 5          | 172       |
| Swap: | 0     | 0    | 0    |        |            |           |

```

Network Usage
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKN
WN group default qlen 1000
    link/ether 02:a9:a3:9a:ad:01 brd ff:ff:ff:ff:ff:ff
    inet 10.5.102.94/16 brd 10.5.255.255 scope global eth0
        valid_lft forever preferred_lft forever
```

softserve

# REFERENCES & SOURCES



<https://www.gnu.org/software/bash/manual/>

softserve



